



Ok, give me detail separate backend architecture and Backend design (Python, reconciliation-first) for this mini app demo, and can handle that 2 csv file

A clean way to present this is: one small but serious “reconciliation service” backend, fully typed and idempotent, that ingests those two CSVs and exposes them via a simple HTTP API.^[1]

Backend architecture (high level)

- **Tech stack**
 - Python 3.x, FastAPI, Pydantic, SQLAlchemy, PostgreSQL (or SQLite dev), pytest.^[1]
 - Decimal everywhere for money; all money columns as NUMERIC(18,4) in DB.^[1]
- **Main layers**
 - **API layer (FastAPI)**
 - Endpoints for importing CSVs, running reconciliation, approving matches, and reading audit logs.
 - **Domain layer**
 - Entities: BankTransaction, Invoice, ReconciliationSuggestion, ReconciliationEntry, AuditEvent.
 - Pure Python services: IngestionService, ReconciliationService, ApprovalService, all deterministic and testable.
 - **Persistence layer**
 - SQLAlchemy models + repositories with explicit transactions and unique constraints to enforce idempotency.
 - **Background job layer**
 - Simple in-process job runner or async task endpoint; designed so it can later be triggered by Pub/Sub with retry-safe semantics.^[1]
- **Key backend properties**
 - **Idempotent ingestion** using an `import_batch_id` and per-row hash; re-upload of the same file does not create duplicates.
 - **Idempotent reconciliation suggestions** via `suggestion_key = hash(txn_id, invoice_id, rule_version)`.
 - **Append-only audit log** for all imports, suggestions, approvals, and exceptions.

- **Adversarial test cases** for duplicate events, bad CSV rows, rounding, and partial/overpayments.

Backend design details (Python, reconciliation-first)

1. Data model (simplified SQLAlchemy)

- `bank_transactions`
 - `id` (PK), `txn_id` (unique), `txn_date`, `account_iban`, `counterparty_name`, `description`, `amount` (Decimal), `currency`, `bank_reference`, `import_batch_id`.
- `invoices`
 - `id` (PK), `invoice_id` (unique), `invoice_date`, `due_date`, `customer_id`, `customer_name`, `amount_gross` (Decimal), `currency`, `status` (open, paid, partially_paid).
- `reconciliation_suggestions`
 - `id` (PK), `suggestion_key` (unique), `txn_id`, `invoice_id`, `rule_version`, `confidence`, `reason`, `status` (pending, approved, rejected).
- `reconciliation_entries`
 - `id` (PK), `txn_id`, `invoice_id`, `amount_applied` (Decimal), `created_at`; unique constraint on (`txn_id`, `invoice_id`) to prevent double booking.
- `audit_events`
 - `id` (PK), `timestamp`, `actor_type` (system | user), `actor_id`, `event_type`, `entity_type`, `entity_id`, `before` (JSON), `after` (JSON), `metadata` (JSON).

Use Pydantic models mirroring these for request/response DTOs with strict types and Decimal fields.

2. Ingestion workflow

Endpoints:

- `POST /imports/bank`
 - Accepts CSV upload or JSON array matching `bank_transactions.csv` header.
 - Steps:
 - Create `import_batch` record.
 - For each row:
 - Validate schema (non-empty `txn_id`, parse date, parse Decimal).
 - If parsing fails (e.g. `NOT_A_NUMBER`), create an `audit_event` with `event_type="import_row_failed"` and skip that row.
 - Upsert by `txn_id`:
 - If new: insert.
 - If existing with identical values: skip (idempotent re-import).

- If existing with different values: log `import_conflict` audit event and skip or flag as exception.
- At end, emit `import_completed` event with summary stats.
- `POST /imports/ar`
 - Same flow but writes into `invoices` with upsert by `invoice_id`.
 - Any unknown status or bad currency leads to `import_row_failed` events.

3. Reconciliation engine

Service: `ReconciliationService.run(rule_version: str)` and endpoint `POST /reconcile/run`.

- Load all:
 - Unmatched `bank_transactions` (no `reconciliation_entries` yet).
 - `invoices` with `status` in ('open', 'partially_paid').
- For each bank transaction:
 - Filter candidate invoices:
 - Same currency.
 - Amount equality within tolerance (e.g. exact match or small rounding diff for tests like 1499.995 vs 1500.00).
 - Date within configurable window ($\pm N$ days).
 - Optional customer name similarity.
 - If exactly one strong candidate:
 - Build deterministic suggestion: `suggestion_key = sha256(f"{txn_id}-{invoice_id}-{rule_version}")`.
 - Upsert into `reconciliation_suggestions`:
 - If suggestion with that key exists: leave as is (idempotent).
 - Else insert with `status="pending"`, `reason`, `confidence`.
 - If multiple candidates:
 - Create suggestions for each with lower confidence and more detailed reason.
 - If no candidates:
 - Optionally log `unmatched_txn` event for later exception handling.
- Emit an `audit_event` of type `reconcile_run_completed` with counts (created, skipped, unmatched).

4. Approval and booking

Endpoint: `POST /reconcile/approve`

- Request body: list of `suggestion_ids` and `actor_id` (user id / email).
- For each suggestion:
 - Load suggestion and associated bank transaction + invoice.
 - Start DB transaction:
 - Check suggestion status == "pending"; otherwise skip/log.
 - Check invoice not fully paid already; if fully paid, log `approval_conflict`.
 - Compute `amount_to_apply`:
 - For demo: exact invoice amount or `min(invoice remaining, abs(bank amount))`.
 - Insert `reconciliation_entry` (enforced by unique constraint (`txn_id`, `invoice_id`)).
 - Update invoice status (paid if fully matched, `partially_paid` if residual).
 - Update suggestion status="approved".
 - Create `audit_event`:
 - `before` = prior invoice snapshot.
 - `after` = new invoice status and remaining balance.
 - `event_type="reconcile_approved"`, `actor_type="user"`.
 - Commit transaction.

This makes double-clicking the same suggestion or replaying the request safe: the unique constraint and status checks prevent a second booking.

5. Audit log and read APIs

Read endpoints:

- `GET /audit`
 - Query params: `actor_type`, `event_type`, `entity_type`, `entity_id`, date range, paging.
 - Returns a paginated list of audit events for the cockpit timeline.
- `GET /bank-transactions`, `GET /invoices`, `GET /reconciliation-suggestions`
 - Support filters for status, date ranges, search by `customer_name`, pagination and sorting.

These power your heavy tables and filters on the Next.js side.

6. Adversarial tests and "Pub/Sub ready" behavior

Using `pytest`:

- **Duplicate CSV rows**
 - Import `bank_transactions.csv` once → N rows.

- Import same file again → still N rows, with `import_completed` showing duplicates skipped and idempotent behavior.
- **Duplicate `txn_id` with different contents**
 - Include both BNK-2026-000015 rows; test that one persists and one generates an `import_conflict` audit event without corrupting data.
- **NOT_A_NUMBER and encoding issues**
 - Ensure invalid rows do not crash the import; they generate `import_row_failed` and are skipped.
- **Reconcile run retries**
 - Run reconciliation twice; verify `reconciliation_suggestions` count does not increase on second run due to `suggestion_key` uniqueness.
- **Approval replay**
 - Approve the same suggestion twice or re-send the same HTTP request; verify only one `reconciliation_entry` is created and a second attempt leads to no new booking but possibly a `duplicate_approval_attempt` event.

All of this supports the story that the service can safely sit behind a Pub/Sub subscription where messages may be retried, dropped, or delivered twice, but money logic never silently goes wrong.^[1]

If you want, next step can be a short FastAPI folder structure (modules and filenames) that you can mirror directly into your repo.

**

1. Plutus-Web-or-App-Development.pdf
2. work.experience_backend_fastapi